

CH5: PARALLEL DATABASE SYSTEMS

Contents

Parallel System Architectures in DDBMS.....	1
Goals of parallel database system	1
Parameters for parallel database	1
1.Shared Memory Architecture	3
2. Shared Disk Architecture	3
3. Shared Nothing Architecture	3
4. Hierarchical Architecture.....	4
5. NUMA (Non-Uniform Memory Access).....	5
6. COMA (Cache-Only Memory Architecture)	5

Parallel System Architectures in DDBMS

- Sometimes centralized systems are not much efficient to handle data with high data transfer rate, due to which we use parallel database systems.
- Parallel system architecture refers to how **processors**, **memory**, and **storage** components are organized and connected to **achieve high performance, reliability, and scalability** in distributed or parallel database environments.
- These architectures define how **multiple processors** coordinate to process transactions or queries over **distributed data** in intercommunication network.

Goals of parallel database system

- **Improved Performance** – Execute queries and transactions faster using multiple processors.
- **High Throughput** – Support many queries and transactions simultaneously.
- **Scalability** – Maintain performance as data volume or number of users grows.
- **Fault Tolerance** – Continue operating smoothly even if some components fail.
- **Load Balancing** – Distribute work evenly across all processing nodes.
- **Efficient Resource Utilization** – Maximize use of CPU, memory, and disk resources.
- **Data Availability** – Ensure continuous access to data across distributed sites.
- **Cost-Effectiveness** – Achieve high performance without needing expensive supercomputers.

Parameters for parallel database

1. **Response Time:** Time taken to complete a single task (query or transaction).

$$\text{Response Time} = \text{Time from query start to finish}$$

◆ Goal: Lower response time = better performance.

◆ Example:

- Query on 1 node: 20 seconds
- Query on 4 parallel nodes: 6 seconds → Response time has **improved** due to parallelism.

2. **Speedup:** How much faster a parallel system performs a task compared to a single-processor system.

$$\text{Speedup}(S_p) = \frac{T_1}{T_p}$$

T_1 = time on 1 processor

T_p = time on p processors

◆ Goal: Higher speedup = better parallel efficiency.

◆ Example:

- Time with 1 processor = 100 sec
- Time with 4 processors = 25 sec

3. **Linear Speedup:** When the speedup is proportional to the number of processors used.

◆ Ideal Condition:

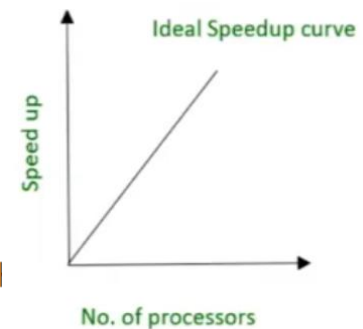
$$S_p = p$$

• S_p : Speedup achieved

• p : Number of processors used.

◆ Example:

- 1 processor → 100 sec
- 2 processors → 50 sec
- 4 processors → 25 sec This shows linear speedup (doubling processors halves time).

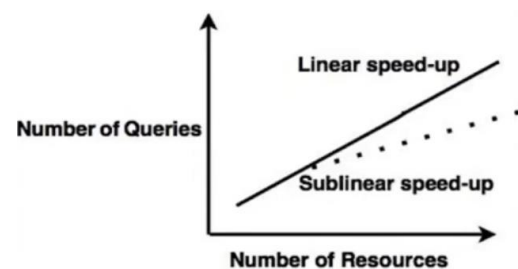


4. **Sub-linear Speedup:** Speedup is less than the number of processors used (common in practice).

◆ Reason: Overheads like communication, synchronization, or load imbalance.

◆ Example:

- 1 processor → 100 sec
- 4 processors → 35 sec
- So $S_4 = 100/35 = 2.86 < 4$ (which is in case of linear)



5. **Scaleup:** Measures a system's ability to handle a proportionally larger workload as more resources are added.

$$\text{Scaleup} = \frac{T_{1,W}}{T_{p,W'}}$$

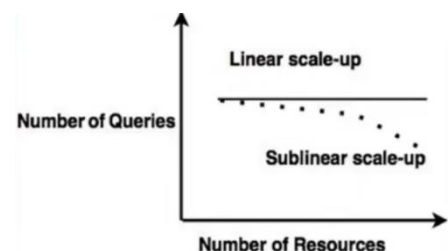
• $T_{1,W}$ = time for workload W on 1 processor

• $T_{p,W'}$ = time for p processors with workload W' , where $W' = p \times W$

◆ Goal: Maintain same response time even if data and processors both scale.

◆ Example:

- 1 processor takes 1 hour for 10 GB
- 4 processors take 1 hour for 40 GB → Good **scaleup**



Parameter	Formula	Interpretation
Response Time	Direct measurement	Time to complete one task
Speedup	$S_p = \frac{T_1}{T_p}$	How much faster p processors are vs 1
Linear Speedup	$S_p = p$	Ideal case
Sub-linear Speedup	$S_p < p$	Real-world case due to overheads
Scaleup	$\frac{T_{1,W}}{T_{p,W'}}$	Efficiency as workload and resources both increase

1. Shared Memory Architecture

Multiple processors share a single memory space and access it through a common bus or interconnect.

- All processors access **global shared memory**.
- Each processor may have its own **cache**.
- Often uses **mutexes or semaphores** for synchronization.

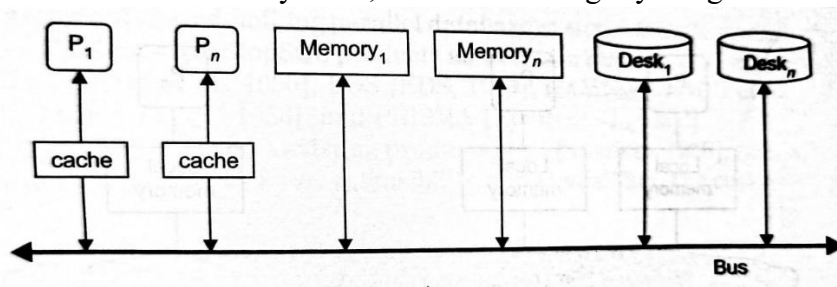
◆ Advantages:

- Easy to program.
- Fast communication between processors.
- Suitable for **tightly coupled systems**.

◆ Disadvantages:

- **Scalability limits** due to memory contention and bus bottlenecks.
- More complex to manage **cache coherence**.

◆ **Use Case:** Small to medium-sized systems; centralized or tightly integrated DDBMS.



2. Shared Disk Architecture

Each processor has its own private memory, but all processors share **common disk storage**.

- Processors communicate through **message passing** or interconnects.
- Disk contains shared data accessible to all nodes.
- Common in **high availability clusters** (like Oracle RAC).

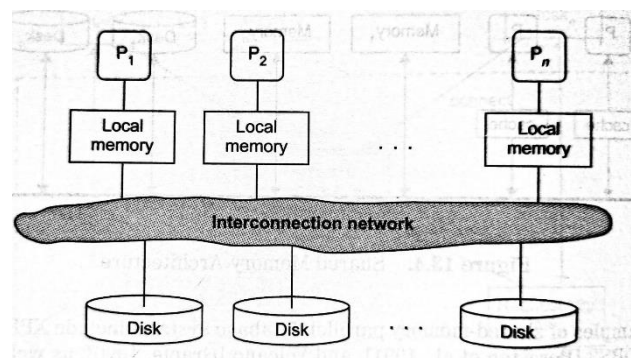
◆ Advantages:

- Easy data sharing.
- High fault tolerance – if one node fails, others can take over.

◆ Disadvantages:

- Disk contention and concurrency control overhead.
- Requires **complex coordination** (e.g., distributed locks).

◆ **Use Case:** Fault-tolerant systems with shared storage, like clustered databases.



3. Shared Nothing Architecture

Each node (processor) has its own **private memory and private disk**. No component is shared.

- Data is **partitioned** across nodes.
- Communication is done via **message passing**.

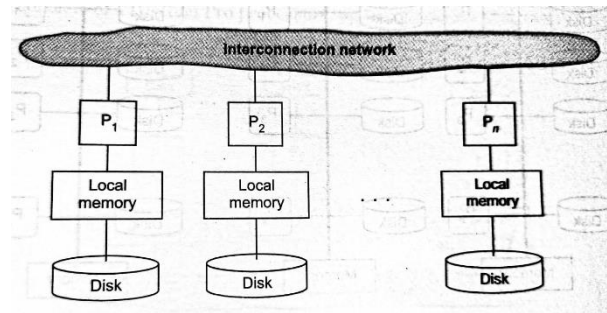
◆ Advantages:

- **Highly scalable** – each node works independently.

- No contention for memory or disk.
- Failure in one node doesn't affect others directly.

◆ **Disadvantages:**

- **Harder to manage** – requires careful partitioning and data distribution.
- Load balancing and fault tolerance are complex.



◆ **Use Case:** Large-scale distributed systems like Google Spanner, Amazon DynamoDB, NoSQL databases.

4. Hierarchical Architecture

Processors and resources are arranged in a **hierarchical tree-like structure**, often based on geography or role.

- Is a combination of shared memory and shared nothing architectures.
- The idea is to build a shared nothing model whose nodes are shared memory.
- Communication flows through levels of the hierarchy.

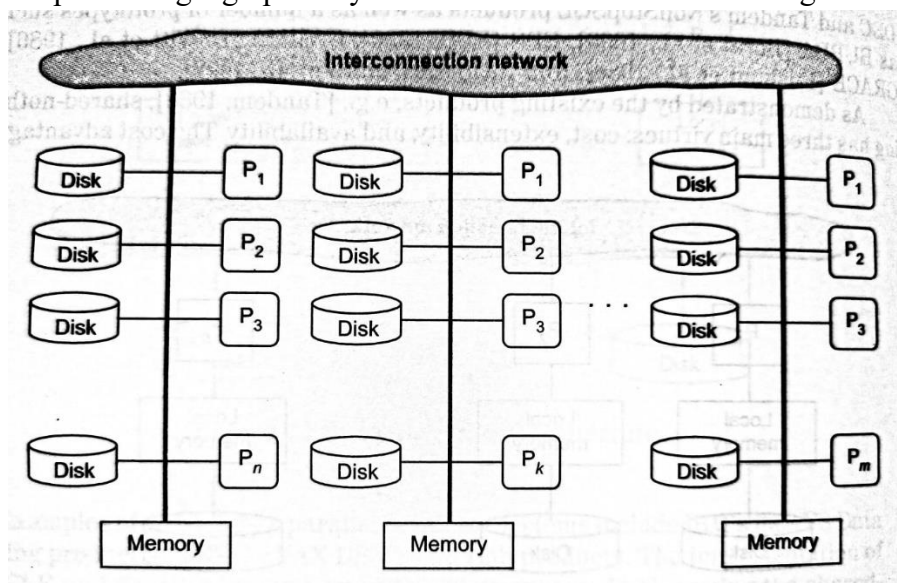
◆ **Advantages:**

- Flexibility and performance of shared memory.
- Secure like shared nothing.
- Easy to manage **large-scale systems**.
- Supports **data localization** and **regional autonomy**.

◆ **Disadvantages:**

- **Single points of failure** at higher levels.
- May introduce **latency** due to level-based communication.

◆ **Use Case:** Corporate or geographically distributed DDBMS with clear regional divisions.



5. NUMA (Non-Uniform Memory Access)

A memory architecture where **each processor has faster access to its local memory** and slower access to memory local to other processors.

- Combines features of shared and distributed memory.
- Maintains **global address space**.
- Memory access time **depends on location**.

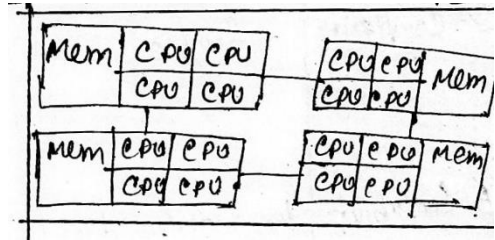
◆ Advantages:

- Good **performance scaling** with more processors.
- Avoids bottlenecks of purely shared memory.

◆ Disadvantages:

- **Programming complexity** increases.
- May suffer performance penalties if memory locality isn't optimized.

◆ **Use Case:** High-performance computing systems, hybrid architectures.



6. COMA (Cache-Only Memory Architecture)

A memory architecture where **all memory acts like a cache** — there is **no fixed home location** for data.

- Data moves dynamically between nodes.
- Cache coherence protocols maintain consistency.

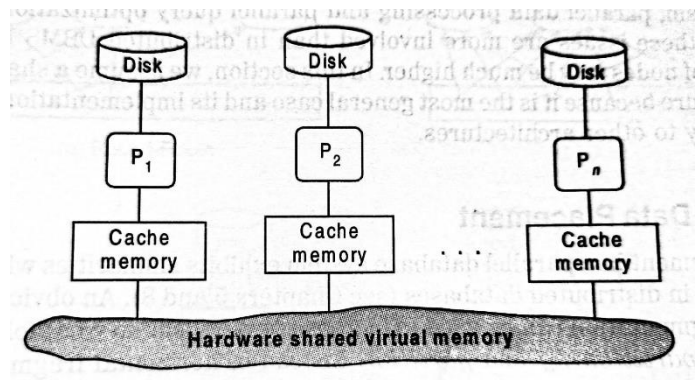
◆ Advantages:

- Efficient memory use — data automatically migrates where needed.
- High flexibility and **data locality**.

◆ Disadvantages:

- Very **complex hardware and coherence protocols**.
- Hard to implement at scale.

◆ **Use Case:** Experimental or specialized parallel systems in research or high-end computing.



Architecture	Shared Memory	Shared Disk	Shared Nothing	Hierarchical	NUMA	COMA
Memory	Shared	Private	Private	Mixed	Distributed	Fully cache
Disk	Shared	Shared	Private	Mixed	Mixed	Distributed
Scalability	Low-Medium	Medium	High	High	High	Medium
Fault Isolation	Low	Medium	High	High	Medium	Medium
Complexity	Low	Medium	High	Medium	High	Very High
Typical Use	Centralized	Clusters	Large DDBMS	Global setups	HPC	Research